

FORMALIZATION

A LANGUAGE MODEL

We introduce our novel language model that is built on top of the core syntaxes of Rust MIR and HIR. We model the core syntaxes of the language model in Backus-Naur Form (BNF) as shown in Table 1. We discuss the formalized syntax and its semantics as follows.

(1) We define basic block $b \in \mathbb{Z}$ as a unit of control flow identified by an integer, consisting of a sequence of statements followed by a terminator. The type τ represents our encoding of Rust type system, encompassing all built-in types (e.g., primitive types, sequence types, algebraic data types, function types, pointer types), generics and trait types, and standard library types. A local variable v , such as v_0, v_1 , is a typed storage location defined inside basic blocks of a function. It can be read, written, moved, or borrowed during execution. A function f consists of a list of arguments, a return value, and a body composed of basic blocks.

(2) The assignment statement ($p = r$) overwrites the value of left-hand expression p with a new value expressed by the right-hand expression r . The expression p is a Place, which can be a local variable v , a pointer dereference ($*v$), a qualified path representing a field access ($v.f$), or an array indexing ($v[i]$). The expression op operand is a value read from a Place, either by copying or moving it. A move operand transfers ownership. The expression r is an Rvalue, which expresses operations performed on operands such as referencing ($\&p$), dereferencing ($*p$), computing the discriminant of the place, and type conversion, *etc.* Note that assignments connect Place with Rvalue. For general Rvalue operations, the Rust type system requires that p and r must have the same type, hence a data flow from r to p can be established. However, when a type conversion is desired, the Rvalue operation *conv* is needed to support assignment operations between different types.

(3) The conversion operation (CastKind) converts values from one type (τ_1) to another type (τ_2). CastKind [4] supports both safe and unsafe type conversions. For example, *IntToFloat* and *FloatToInt* conversions are safely handled by the Rust compiler. We specifically retain the unsafe type conversion in CastKind, namely the *As* and *Transmute* operations. The *As* operation represents raw pointer casting, including *PtrToPtr* and *FnPtrToPtr*.

(4) We define several types of terminators. Function call (*Call*($f, [op_1, \dots], (p, b)$)) invokes a function f with arguments $[op_1, \dots]$. The return value is assigned to p in the basic block b . *Goto* (*Goto*(b)) unconditionally jumps to the successor basic block b . *Return* exits from the current function. *SwitchInt* (*SwitchInt*($op, [b_1, b_2, \dots]$)) directs control flow to one of the basic blocks in the target list $[b_1, b_2, \dots]$ conditionally on the value of the discriminant operand op . By indexing the target list using the value of the discriminant operand op at runtime, the control flow jumps to the target basic block. It matches the *if* and pattern matching constructs in Rust.

B ABSTRACT DOMAIN

We adopt a lattice scheme to achieve efficient and scalable data-flow analysis. We employ the Tarjan algorithm on the control-flow graphs (CFGs) to separate the strongly connected components (SCCs). Each SCC contains a set of nodes representing basic blocks.

Table 1: Core syntax of DERUST's language model.

BasicBlock	b	$\in$	$\mathbb{Z}$
Type	τ	$::=$	Bool, Char, Int, Uint, Float, Adt, Str, Array, Slice, Tuple, RawPtr, Ref, Param, ... (Built-in types) Generic type Standard library types
Local	v	$\in$	$\{v_0, v_1, v_2, \dots\}$
Function	f	$\in$	$\{f_0, f_1, f_2, \dots\}$
Place	p	$::=$	$v \mid *v \mid v.f \mid v[i]$
Operand	op	$::=$	copy $p \mid$ move p
CastKind	$conv$	$::=$	As Transmute
Rvalue	r	$::=$	$op \mid \&p \mid *p \mid conv(p, \tau_1, \tau_2) \mid disc(p, f)$
Statement	s	$::=$	$p = r$
Terminator	t	$::=$	$Call(f, [op_1, op_2, \dots], (p, b)) \mid Goto(b) \mid Return \mid SwitchInt(op, [b_1, b_2, \dots])$

Statements in one node are analyzed sequentially, which is flow-sensitive. We define the abstract domain to represent aliasing relationships between program variables. We define the abstract state for a single variable as AState, which represents the set of program variables it may alias with at a given program point. Formally:

$$AState = \mathcal{P}(\text{Vars})$$

We construct the lattice structure with the following components:

$$\langle AState, \sqsubseteq, \sqcup, \perp, \top \rangle$$

where:

- The bottom element $\perp$: maps every variable v to itself, representing the initial state where no aliasing information is known, meaning that the variable is only self-aliased.
- The top element $\top$: maps every variable to Vars, representing the most imprecise state where any variable may alias with all others.
- The partial order: For any two $AState_1, AState_2$,

$$AState_1 \sqsubseteq AState_2 \iff AState_1 \subseteq AState_2$$

- The join operation: For any two $AState_1, AState_2$,

$$AState_1 \sqcup AState_2 = AState_1 \cup AState_2$$

The abstract state at each program point is defined as a mapping from all the variables Vars to their respective AStates.

C TRANSFER FUNCTIONS

The transfer functions are defined as the set of instructions that can potentially change the alias states. We employ a unified alias state update strategy for each instruction. When we assign the Rvalue (e.g., variable b) to the left Place (e.g., variable a), they become aliases. We define several assignment statements in our language model, including general assignments (e.g., $a=b$, $a=\text{move } b$), reference assignments (e.g., $a=\&b$), AddressOf assignments (e.g., $a=\&b$ as $*const \mid *mut$), dereference assignments, type conversion assignments, *etc.* We also define terminators such as function call

(e.g., $a = \text{Fn}(b)$). Despite their different semantics, we uniformly apply the aliasing update rule ($S_a = S_a - a, S_b = S_b \cup a$), where S_i represents the set of aliases of variable i . Furthermore, due to the well-known challenges of pointer analysis, we approximate the alias relationships by disregarding the pointer arithmetic.

We formalize the transfer functions using inference rules over abstract states. Each rule captures a instruction that modifies aliasing information based on the language model. For simplification, let $\mathcal{A}$ be the abstract state AState, where each variable v is mapped to an alias set $S_v = \mathcal{A}(v)$. The symbol r denotes the initial Rvalue and serves as the basis for analysis in the transfer rules. We define transfer rules for the following representative instructions:

$$\frac{r = b \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = b \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{ASSIGN}]$$

$$\frac{r = \text{move } b \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = \text{move } b \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{MOVE}]$$

$$\frac{r = \&b \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = \&b \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{REF}]$$

$$\frac{r = \&b \text{ as } * \text{const} | * \text{mut} \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = \&b \text{ as } * \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{ADDR OF}]$$

$$\frac{r = *b \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = *b \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{DEREF}]$$

$$\frac{r = \text{conv}(b, \tau_1, \tau_2) \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = \text{conv}(b, \tau_1, \tau_2) \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{CAST}]$$

$$\frac{r = \text{Call}(f, [b]) \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = \text{Call}(f, [b]) \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{CALL-C}]$$

$$\frac{r = \text{Call}(f, [\text{move } b]) \quad S_a = \mathcal{A}(a) \quad S_b = \mathcal{A}(b)}{\mathcal{A} \vdash a = \text{Call}(f, [\text{move } b]) \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b \mapsto S_b \cup \{a\}]} \quad [\text{CALL-M}]$$

$$\frac{r = b.f \quad S_a = \mathcal{A}(a) \quad S_{b.f} = \mathcal{A}(b.f)}{\mathcal{A} \vdash a = b.f \Rightarrow \mathcal{A}[a \mapsto S_a \setminus \{a\}, b.f \mapsto S_{b.f} \cup \{a\}]} \quad [\text{FIELD ASSIGN}]$$

For algebraic data types, we apply a field-sensitive approach to enhance the precision. We maintain the alias state for each subfield variable. During the execution of each instruction, when an alias set is updated, changes are synchronized to the alias sets of its subfields. Conversely, changes in a subfield do not update the parent alias state.

D FIXED-POINT ALGORITHM

We implement a fixed-point algorithm to update the abstract states until a fixed point is reached. We adopt the worklist algorithm, where the worklist W is initialized to contain all the SCCs in the CFG. For each SCC C in W , the current abstract state $\mathcal{A}_C$ is first computed by *joining* the abstract states of all its predecessor SCCs:

$$\mathcal{A}_C := \bigsqcup_{P \in \text{pred}(C)} \mathcal{A}_P$$

We then iteratively apply the transfer functions $\mathcal{F}$ to each basic block $b \in C$ in a topologically sorted order, and update the alias state accordingly:

$$\mathcal{A}_C := \mathcal{F}_b(\mathcal{A}_C), \quad \forall b \in C$$

If any update in $\mathcal{A}_C$ causes the state to go upward in the lattice (i.e., $\mathcal{A}_C$ is strictly changed), all successor SCCs $\text{succ}(C)$ are reinserted into the worklist W for further propagation and re-analysis.

The algorithm terminates when W becomes empty, indicating that a fixed point has been reached for all program points. Correctness is guaranteed by the monotonicity of the transfer functions and the finite height of the lattice.

$$\frac{\forall P \in \text{pred}(C), \mathcal{A}_C^{\text{in}} := \bigsqcup \mathcal{A}_P \quad \forall b \in C, \mathcal{A}_C^{\text{out}} := \mathcal{F}_b(\mathcal{A}_C^{\text{in}})}{\text{If } \mathcal{A}_C^{\text{out}} \neq \mathcal{A}_C^{\text{prev}}, \text{ then enqueue } \text{succ}(C) \text{ into } W} \quad [\text{FIXPOINT-STEP}]$$

E INTER-PROCEDURAL ANALYSIS

To support modularity and avoid redundant recomputation, we design a summary-based, context-sensitive inter-procedural analysis. Each function invocation is resolved using a memoized lookup table:

$$\text{Cache}[(f, \text{in_state})] = \text{out_state}$$

f represents the function name, in_state represents the alias states of the function arguments, and out_state is the function summary. The function summary contains two types of information. First, it captures the alias relationships among function arguments and the return value. This can be used to update the alias state of the caller function in a field-sensitive manner. Second, the function summary tracks the occurrences of unsafely converted values within function arguments and return value.

When analyzing a call instruction $a = f(\text{args})$, the analysis checks whether the current calling context $(f, \text{in_state})$ already exists in the cache. If so, the summary is directly applied to the caller state by updating the alias set of the return variable a , and propagating field-sensitive alias effects to the arguments:

$$\frac{(f, \text{in_state}) \in \text{Cache} \quad \text{Cache}[(f, \text{in_state})] = \text{out_state}}{\mathcal{A} \vdash a = f(\text{args}) \Rightarrow \mathcal{A}[a \mapsto \text{AliasRet}(\text{out_state}), \text{Update}(\text{args}, \text{out_state})]} \quad [\text{CALL-SUMMARY}]$$

Otherwise, if no summary is found, the callee function f is recursively analyzed under the current in_state , producing a new output state that is cached for future use:

$$\frac{(f, \text{in_state}) \notin \text{Cache} \quad \text{out_state} = \text{Analyze}(f, \text{in_state})}{\mathcal{A} \vdash a = f(\text{args}) \Rightarrow \text{Cache}[(f, \text{in_state})] := \text{out_state}, \mathcal{A}[a \mapsto \text{AliasRet}(\text{out_state}), \text{Update}(\text{args}, \text{out_state})]} \quad [\text{CALL-RECURSE}]$$